

Project Plan: Energy-Aware Compiler Optimization for C/C++

Integrated Semantic Analysis, IR Generation, and Optimization

August 23, 2026

Abstract

This project investigates whether combining semantic analysis, intermediate-representation (IR) generation, and selected code-optimization passes can reduce the energy and time used to compile C and C++ programs. The proposed compiler uses one traversal of the abstract syntax tree (AST), produces typed IR incrementally, and performs safe local optimizations as IR instructions are emitted. The first part of this plan defines a repeatable method for measuring compilation energy. The second part gives the architecture and implementation steps for merging the three stages.

1 Project Objective and Scope

Primary objective: minimize the energy consumed by the *compiler process* while preserving language correctness, useful diagnostics, and generated-code quality.

Secondary objective: verify that reducing compiler work does not increase the runtime energy of the compiled program. The system will therefore report two different quantities:

- **Compilation energy** E_{compile} : joules consumed while translating a C/C++ source program into an object file or executable.
- **Program energy** E_{run} : joules consumed when the resulting executable runs a fixed workload.

The initial prototype should support a clearly documented subset of C, followed by C++ features in controlled increments. Full C++ semantic analysis is complex; templates, overload resolution, exceptions, and constant evaluation should not all be attempted in the first milestone.

2 Part I: Energy Calculation Steps

2.1 Measurement boundaries

Use the same machine, operating-system configuration, compiler build, command line, source files, and input data for every comparison. Define two compiler configurations:

1. **Baseline:** conventional separate semantic-analysis, IR-generation, and optimization stages.
2. **Merged:** integrated semantic analysis, typed-IR emission, and safe online optimization.

Measure from immediately before compiler-process creation until the process exits. Report front-end-only results separately from complete compilation and linking. This prevents linker and file-system activity from hiding the effect of the merged stages.

2.2 Step-by-step energy calculation

Step 1: Prepare the machine. Disable unrelated background work, use a fixed power policy and CPU frequency policy, connect AC power, record the CPU model, and allow the machine to reach a stable temperature. Keep these settings identical across experiments.

Step 2: Select the energy source. Prefer CPU package/DRAM energy counters exposed by hardware (for example, RAPL-compatible counters). If unavailable, use an external power meter. A software power estimate may be used only when its limitations are reported.

Step 3: Measure idle power. With the machine in the same state but no compilation running, sample power for T_{idle} seconds:

$$P_{\text{idle}} = \frac{E_{\text{idle}}}{T_{\text{idle}}}.$$

Step 4: Warm up. Run one unrecorded compilation to stabilize executable pages, shared libraries, and CPU state. Decide whether the study represents a warm-cache or a cold-cache workload, and use that policy consistently.

Step 5: Record counter values. For trial i , record energy immediately before and after compilation, together with wall-clock time:

$$E_{\text{raw},i} = E_{\text{after},i} - E_{\text{before},i}, \quad T_i = t_{\text{after},i} - t_{\text{before},i}.$$

If a finite-width counter wraps, add its documented maximum range before subtracting.

Step 6: Subtract idle energy. Estimate the energy attributable to compilation:

$$E_{\text{compile},i} = E_{\text{raw},i} - P_{\text{idle}}T_i.$$

Also retain raw package energy because idle subtraction can add error to very short runs. Compile a source file repeatedly in one measured batch when a single run is too short.

Step 7: Repeat and summarize. Perform at least 20 interleaved trials per configuration (baseline, merged, baseline, merged, and so on). Report the median, interquartile range, arithmetic mean, and a confidence interval. Interleaving limits bias from temperature and system drift.

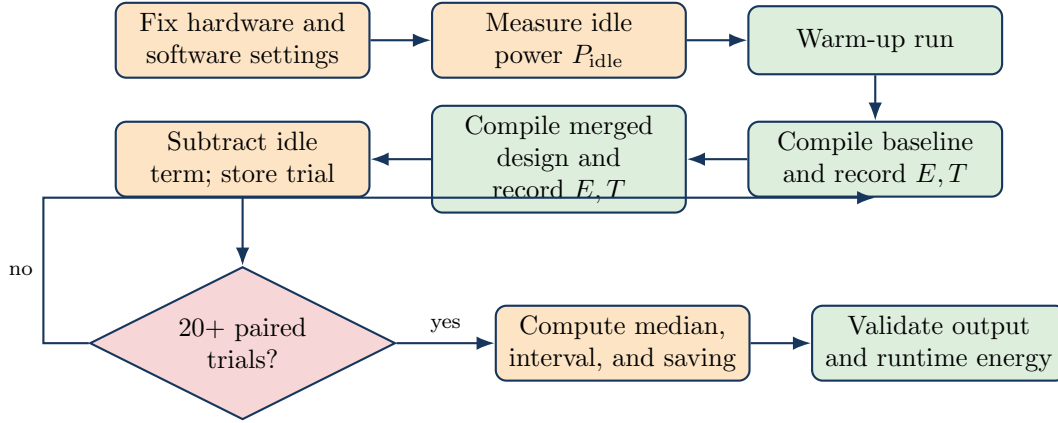
Step 8: Calculate energy reduction. For the same benchmark and compiler output, compute

$$\text{Saving}(\%) = \frac{\tilde{E}_{\text{baseline}} - \tilde{E}_{\text{merged}}}{\tilde{E}_{\text{baseline}}} \times 100,$$

where $\tilde{E}$ is the median adjusted energy. Also report the speedup $T_{\text{baseline}}/T_{\text{merged}}$ and peak memory reduction.

Step 9: Check output quality. Compile and run the same test suite with both compilers. Compare diagnostics, object-code correctness, executable size, execution time, and E_{run} . An energy saving is accepted only if correctness is unchanged and runtime quality stays within a predeclared tolerance.

2.3 Energy measurement flowchart

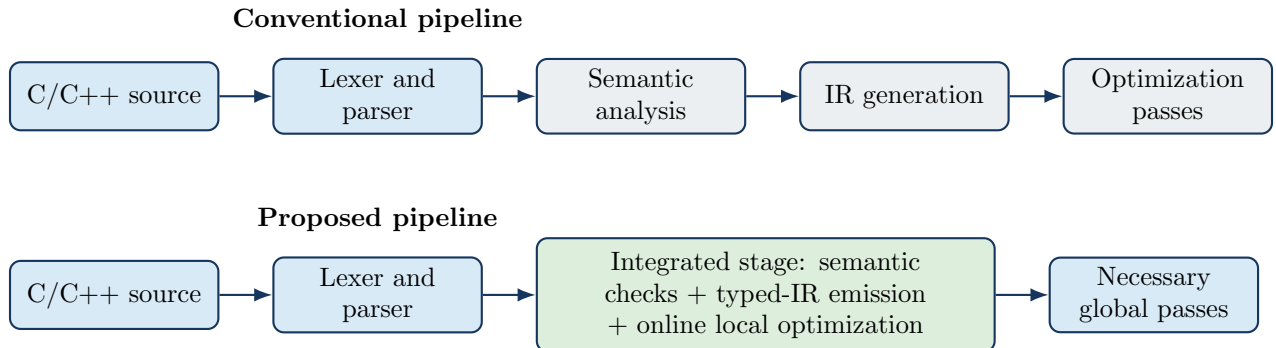


2.4 Required measurements

Metric	How to obtain it	Why it matters
Package/CPU energy (J)	Hardware energy counter before and after the compiler process	Primary compilation-energy result
DRAM energy (J), if available	Memory-domain energy counter	Tests whether fewer AST/IR passes reduce data movement
Wall time (s)	Monotonic clock around compilation	Explains energy change and gives throughput
CPU cycles/instructions	Performance counters	Reveals eliminated compiler work
Cache misses/bytes allocated	Performance counters and compiler instrumentation	Evaluates the benefit of fewer traversals and temporary objects
Peak resident memory	Operating-system process statistics	Detects lower live-state and allocation pressure
Output correctness/quality	Tests, executable time, size, and runtime joules	Prevents savings caused by weaker optimization

3 Part II: Steps for Merging the Compiler Stages

3.1 Conventional and proposed pipelines



The stages are merged operationally, not erased conceptually. Semantic rules must still be enforced, and transformations that require whole-function or whole-program information remain later passes. The key change is to avoid repeatedly walking large structures and constructing IR that an immediately following pass would discard.

3.2 Merging steps

- Stage 1: Define a common typed representation.** Give every expression node a type, value category, source location, and optional constant value. Define a compact typed IR in static single assignment (SSA)-friendly form. Specify invariants and error-recovery rules before implementation.
- Stage 2: Build the semantic environment.** Implement nested symbol tables, scopes, declarations, type compatibility, implicit conversions, function lookup, and control-flow legality. Cache canonical types and identifiers to reduce repeated comparisons and memory.
- Stage 3: Combine checking and emission.** Replace “check subtree, then revisit subtree to emit IR” with a routine that returns a typed value:

$$\text{lower}(\text{AST node, context}) \longrightarrow (\text{type, IR value, effects}).$$

Each visit resolves names and types, diagnoses errors, applies required conversions, and emits IR only when the construct is valid.

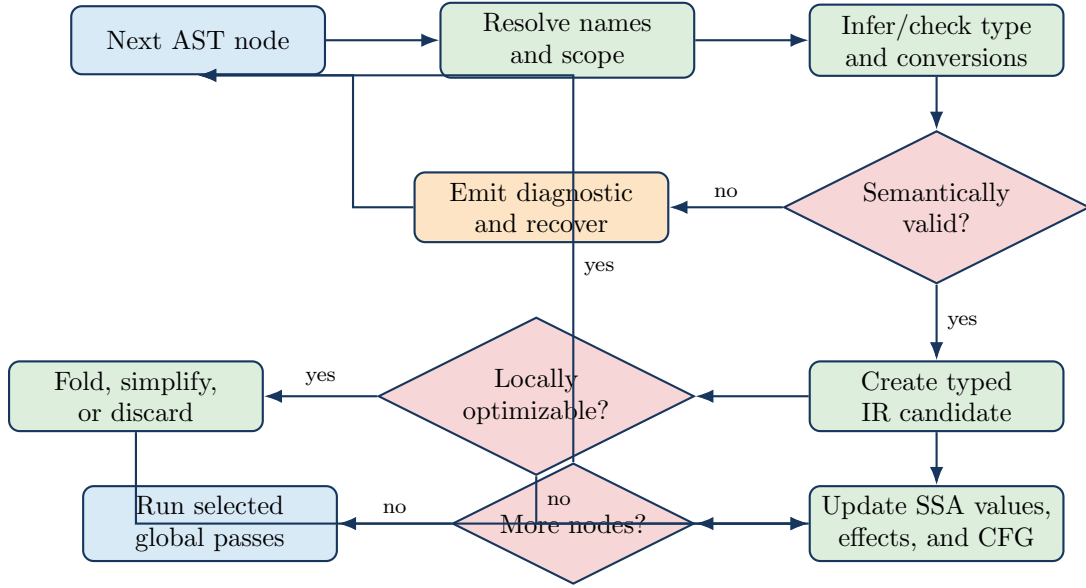
- Stage 4: Optimize during emission.** Apply transformations that need only local facts: constant folding, algebraic simplification, constant propagation within the current scope, dead temporary elimination, redundant cast removal, strength reduction, and unreachable branch removal. Use an IR builder that simplifies an instruction before allocating it.
- Stage 5: Track effects and dependencies.** Attach flags for reads, writes, volatility, atomic operations, exceptions, and possible aliasing. These prevent unsafe reordering or deletion. In C/C++, signed overflow rules, sequence rules, volatile access, and undefined behavior require conservative handling.
- Stage 6: Form basic blocks incrementally.** Create control-flow edges as statements are lowered. Seal blocks when all predecessors are known, insert or simplify ϕ nodes, and immediately discard unreachable blocks when language rules permit.
- Stage 7: Retain a small late optimizer.** Run only analyses that require complete control-flow or cross-function knowledge: global value numbering, loop optimization, vectorization, interprocedural analysis, register allocation, and target-specific code generation. The project should measure which passes are worth their energy cost.
- Stage 8: Add an energy-aware pass policy.** Estimate each optional pass’s benefit and cost using program features and previous measurements. Run a pass only when

$$\hat{B}_p > \lambda \hat{E}_p + \mu \hat{T}_p,$$

where $\hat{B}_p$ is predicted output benefit, $\hat{E}_p$ is compiler energy, $\hat{T}_p$ is time, and λ, μ express the selected compilation budget.

- Stage 9: Validate after every increment.** Differentially test against the baseline, compare diagnostics and IR, execute test programs, run sanitizers on the compiler, and then repeat the energy experiment. Do not combine several transformations before determining which one caused an energy or correctness change.

3.3 Integrated-stage flowchart



4 Energy-Reduction Plan

Action	Expected mechanism	Evidence to collect
One AST traversal	Removes repeated dispatch, pointer chasing, and cache misses	Traversal count, cycles, last-level cache misses
Typed IR on demand	Avoids separate annotations and invalid/transient IR	Allocated bytes, peak memory, IR instruction count
IR-builder simplification	Never materializes obvious redundant operations	Eliminated instruction count and joules
Arena allocation/interning	Reduces allocator calls and duplicate type/name objects	Allocation count and memory footprint
Incremental data-flow facts	Reuses facts already found during semantic checking	Analysis time and repeated-query count
Energy-aware pass selection	Skips expensive passes with low predicted benefit	Per-pass joules versus output improvement
Parallelism threshold	Avoids thread-launch and synchronization cost on small files	Joules/source line at different input sizes
Reduced diagnostics overhead	Lazily formats messages only when needed	Time and allocations on successful builds

The compiler should expose three modes: **eco** (lowest compilation energy), **balanced** (default trade-off), and **performance** (maximum generated-program speed). The same correctness rules apply in all modes; only optional optimizations change.

5 Experimental Plan and Success Criteria

Use small feature tests, medium open-source translation units, and large C/C++ projects. Separate C and C++ results, and group programs by size and language features. For each input, compare the baseline and

merged compiler using identical optimization goals.

A successful prototype should satisfy all of the following:

- pass the semantic, diagnostic, IR, and execution regression suites;
- reduce median front-end energy by at least 10% on the target benchmark set;
- reduce or preserve median wall time and peak memory;
- keep executable runtime, runtime energy, and size within 2% of the baseline in balanced mode; and
- show statistically stable results across repeated, interleaved measurements.

6 Implementation Schedule

Weeks	Work	Deliverable
1–2	Benchmark harness, counter reader, idle correction, and baseline measurements	Reproducible energy report
3–4	Typed IR specification, semantic environment, and C subset	Verified semantic front end
5–7	Integrated AST lowering and IR builder	Single-pass typed IR generation
8–9	Online folding, simplification, CFG/SSA construction, effect tracking	Merged-stage prototype
10–11	Energy-aware pass selection and remaining global passes	Eco/balanced/performance modes
12	Differential testing, energy evaluation, and final analysis	Results, limitations, and recommendations

7 Main Risks and Controls

- **Incorrect early optimization:** require typed IR invariants, effect tracking, conservative alias rules, and differential tests.
- **Poor diagnostics after fusion:** retain source locations and a recoverable error value so IR emission can stop locally without losing later diagnostics.
- **Higher implementation complexity:** use one public lowering interface and keep global transformations as separate passes.
- **Noisy energy data:** use long enough batches, paired interleaved trials, idle correction, temperature control, and confidence intervals.
- **Compile-energy savings but slower output:** measure E_{compile} and E_{run} separately and enforce the output-quality threshold.

8 Final Deliverables

The project will produce: (1) a baseline and a merged C/C++ compiler pipeline, (2) an energy measurement harness, (3) per-stage and per-pass energy/time/memory profiles, (4) a regression and benchmark suite, and (5) an evaluation showing when stage fusion saves energy and when a traditional late optimization pass remains worthwhile.